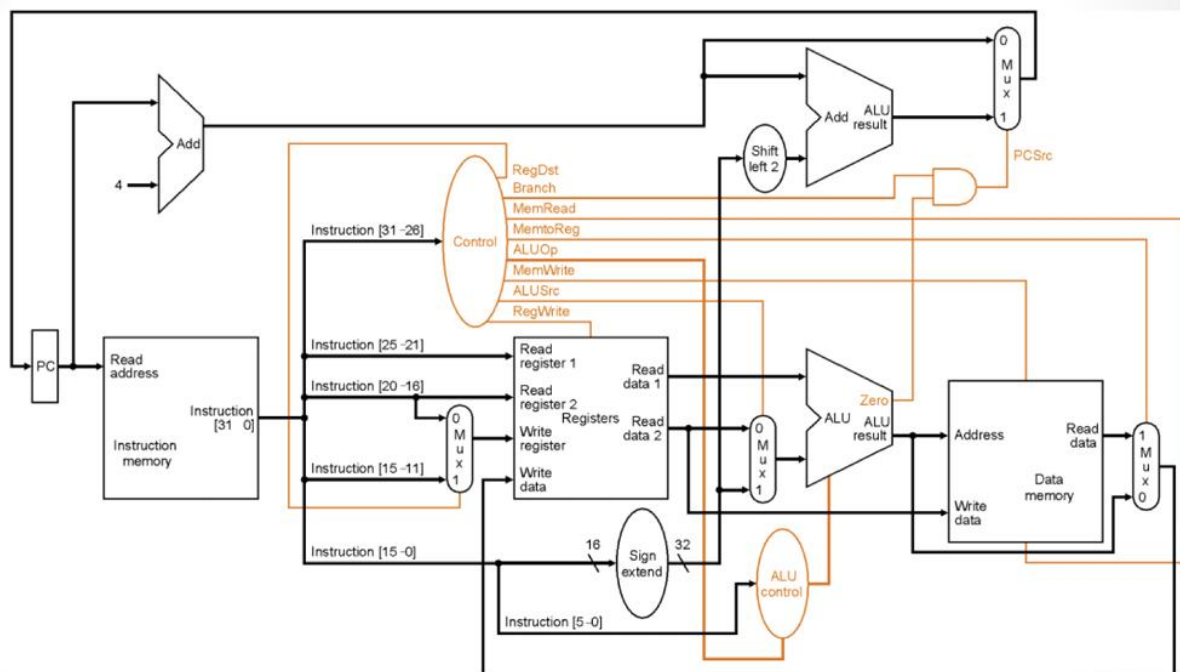


Prepared for: midterm02,
computer architecture class, term
4042.

Prepared by: Zeinab Azordeh
STUDENT CODE: 40317653

Multicycle Processor



MODULES

REGISTER FILE

1. CODE:

To code a register file, we need two read address inputs, and one write address input. One clock and one flag for write enable. Then to be able to read, we need 2, 8_bit data read outputs. Inside the module we add an array that holds all 8 8_bit registers and initialize them to 0.

```
module RegisterFile(  
    input clk,  
    input reg_write_en,  
    input [2:0] read_reg1,  
    input [2:0] read_reg2,  
    input [2:0] write_reg,  
    input [7:0] write_data,  
    output [7:0] read_data1,  
    output [7:0] read_data2  
);  
  
    // Array of 8 registers, each 8 bits wide  
    reg [7:0] registers [7:0];  
  
    integer i;  
  
    // Initialize all registers to 0  
    initial begin  
        for (i = 0; i < 8; i = i + 1) begin  
            registers[i] = 8'b0;  
        end  
    end
```

```
//r0 is always 0. enforce this by returning 0 if read_reg is 0.  
assign read_data1 = (read_reg1 == 3'b000) ? 8'b0 : registers[read_reg1];  
assign read_data2 = (read_reg2 == 3'b000) ? 8'b0 : registers[read_reg2];  
  
// Write port  
always @(posedge clk) begin  
    // Only write if reg_write is high AND we are not trying to write to r0  
    if (reg_write_en && write_reg != 3'b000) begin  
        registers[write_reg] <= write_data;  
    end  
end
```

We enforce r0 to always contain 0 by assigning any read data with address 0 to be 0.

And write only to nonzero addresses when writing is enabled.

WAVE FORM:

Test 1: The Register 0 (R0) Check

Register 0 is special. It should always hold the number zero and cannot be changed.

- It tries to save the number 255 into R0.
- **Result:** It expects to see a **0**. This proves the system correctly ignored the change.

Test 2: Basic Write and Read (R4)

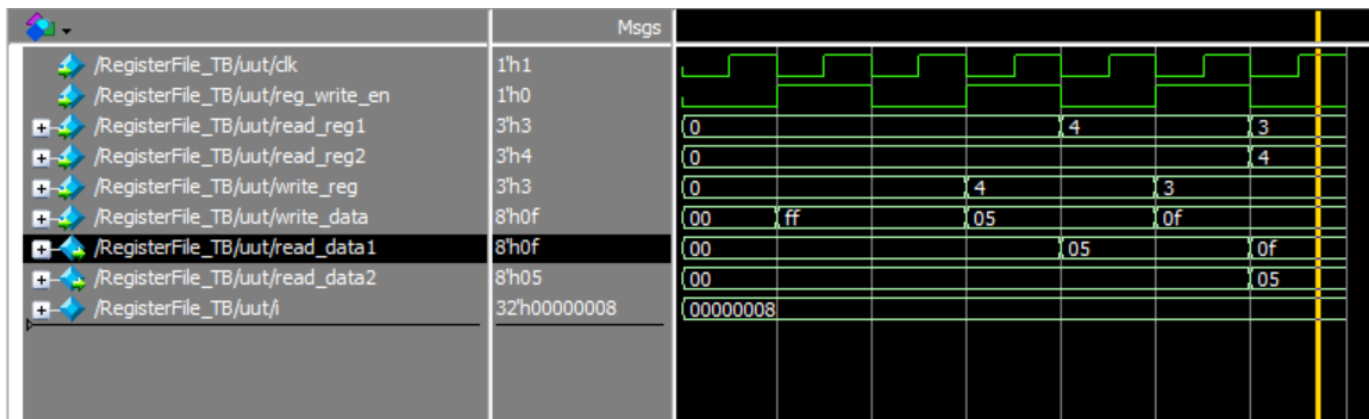
This test checks if a normal register can store a number and give it back.

- It saves the number 5 into Register 4 (R4).
- **Result:** It expects to see a **5**. This proves that basic data storage works.

Test 3: Read Two Registers at Once

This test checks if the system can read two different registers at the exact same time.

- It saves the number 15 into Register 3 (R3). (R4 still has the 5 from Test 2).
- It reads R3 and R4 at the same time.
- **Result:** It expects **15** from R3 and **5** from R4. This proves the system handles simultaneous reads perfectly.



ALU

CODE:

```
always @(*) begin
    case(alu_control)
        3'b000: {c_flag, result} = in1 + in2; // ADD
        3'b001: {c_flag, result} = in1 + in2 + carry_in; // ADDC
        3'b010: {c_flag, result} = in1 - in2; // SUB
        3'b011: {c_flag, result} = in1 - in2 - carry_in; // SUBC
        3'b100: begin result = in1 & in2; c_flag = 1'b0; end // AND
        3'b101: begin result = in1 | in2; c_flag = 1'b0; end // OR
        3'b110: begin result = in1 ^ in2; c_flag = 1'b0; end // XOR
        default: {c_flag, result} = 9'b0;
    endcase
end

// The Zero flag is 1 when the result is exactly 0
assign z_flag = (result == 8'b0);
```

Alu is all combinational logic and holds no clock. It operates 7 required operations and computes z and c flags. To do so we need 2 8_bit inputs, one bit carry input, one result output, and two one bit flags as outputs.

WAVEFORM:

Test 1: The Overflow Test (Addition)

This test checks what happens when a math answer is too big to fit in the normal space.

- It adds 255 and 1 (The system's normal space can only hold numbers up to 255).
- **Result:** It expects the main result to show **0**, but it turns on carry signal to **1**.

Test 2: The Zero Test (Subtraction)

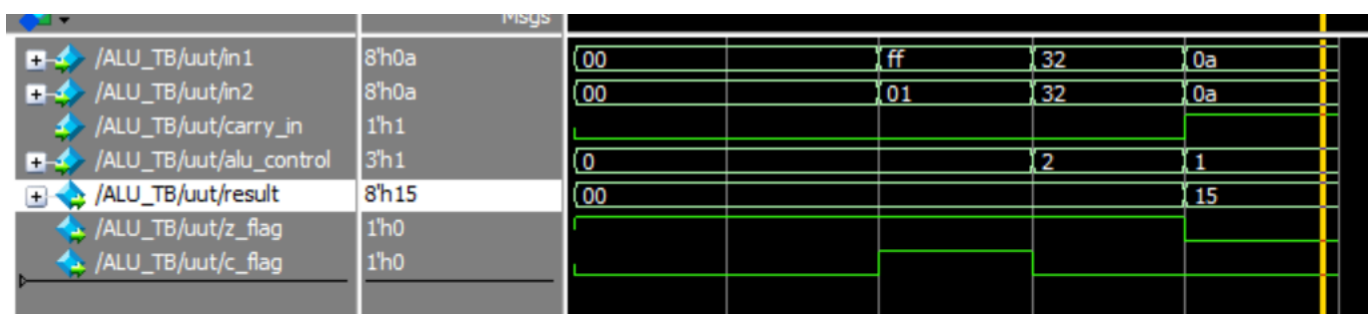
This test checks if the system correctly notices when a math answer is exactly zero.

- **Action:** It subtracts 50 from 50 (final answer must be 0 and z_flag is 1).
- **Result:** It expects the main answer to be **0**, which successfully turns on the special "Zero" signal (**1**).

Test 3: Adding with an Extra Carry

This checks the ADC.

- **Action:** It adds 10 + 10, and also adds an extra 1 (the carry-in).
- **Result:** It expects the final answer to be exactly **21**.



INSTRUCTION MEMORY

CODE:

```
module InstructionMemory(  
    input [11:0] address,      // 2^12 = 4096  
    output [18:0] instruction // 19-bit instruction width  
);  
  
    // Array of 4096 words, each 19 bits wide  
    reg [18:0] rom [0:4095];  
  
    assign instruction = rom[address];  
endmodule
```

Instructions are 19-bit words, and we have 4096 of them. This code is simply one two-dimensional array that is always accessible. Later we will create one program.text that will link to this code.

DATA MEMORY

CODE:

```
// Array of 256 bytes, each 8 bits wide  
reg [7:0] ram [0:255];  
  
// Write  
always @(posedge clk) begin  
    if (mem_write) begin  
        ram[address] <= write_data;  
    end  
end  
  
// Read ( make it continuous based on mem_read)  
always @(*) begin  
    if (mem_read) begin  
        read_data = ram[address];  
    end else begin  
        read_data = 8'b0; // Default zero  
    end  
end
```

In this module we will return to holding our clock. It is not much more than the instruction memory, just has some flags and data to write.

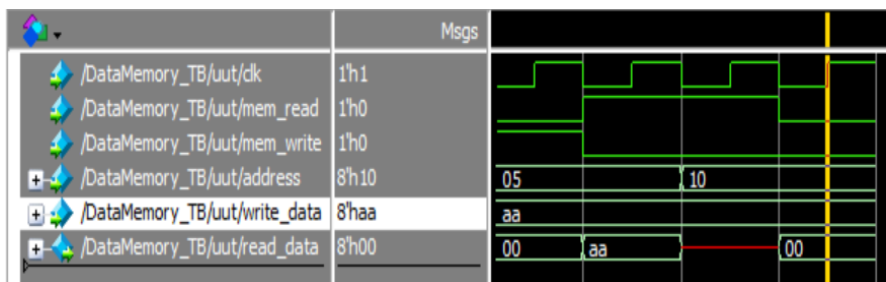
Mem_read and mem_write are enable keys.

Write_data is what we are gonna write.

WAVEFORM:

Test 1: Saving and Recalling

We put the value AA into a specific memory slot and immediately read it back. We expect to see aa, proving the memory actually stores information correctly.



Test 2: Checking an Empty Slot

We peek into a memory slot we haven't touched yet. This just shows us what the system outputs when it looks at an empty, uninitialized space.

Test 3: Turning It Off

We flip the memory's "read" switch completely off. The output should drop straight to **00**, proving the system stops sending data when we tell it to.

CONTROL UNIT

CODE:

```
// Datapath Control Signals
output reg PCWrite,
output reg IRWrite,
output reg RegWrite,
output reg MemRead,
output reg MemWrite,

// Multiplexer & ALU Controls
output reg [2:0] ALUControl,
output reg ALUSrc, // 0: 2nd operand is Reg2, 1: 2nd operand is Immediate
output reg MemToReg, // 0: Save ALU result to Reg, 1: Save Mem data to Reg
output reg PCSrc // 0: PC = PC + 1, 1: PC = Branch/Jump Target
```

The control unit, just like the first project, is a boss to set the right signals the right time.

The obvious inputs are clock, reset, instruction and two flags. Outputs must be signals that control other modules.

```
wire [1:0] op_2bit = instruction[18:17];
wire [2:0] op_3bit = instruction[18:16];
wire [4:0] op_5bit = instruction[18:14];

wire [2:0] fn_reg_reg = instruction[16:14];
wire [2:0] fn_reg_imm = instruction[16:14];
wire [1:0] fn_mem = instruction[15:14]; //
wire [1:0] fn_branch = instruction[15:14];
```

We have 5 stages in each multicycle processor (though the decode stage is all formality). Also, we break down the instruction to all meaningful cuts. Op 2, 3, and 5 bits and some other meaningful ones that are explained in the comment section of the code.

For the **FETCH** part we create one always-block on reset or clock, that creates the start of the instruction, the rest of the instruction is handled logically. Then we have one huge switch case situation. Current state can be fetch, decode, execute, memory access, and write backs.

```
FETCH: begin
    IRWrite = 1; // Save instruction to Instruction Register
    PCWrite = 1; // PC = PC + 1
    PCSrc = 0;
    next_state = DECODE;
end

DECODE: begin
    next_state = EXECUTE;
end

MEM_ACCESS: begin
    if (fn_mem == 2'b01) begin // LDM (Load)
        MemRead = 1;
        next_state = WRITEBACK;
    end
    else if (fn_mem == 2'b10) begin // STM (Store)
        MemWrite = 1;
        next_state = FETCH; // Store is complete, fetch next instruction
    end
    else begin
        next_state = FETCH;
    end
end

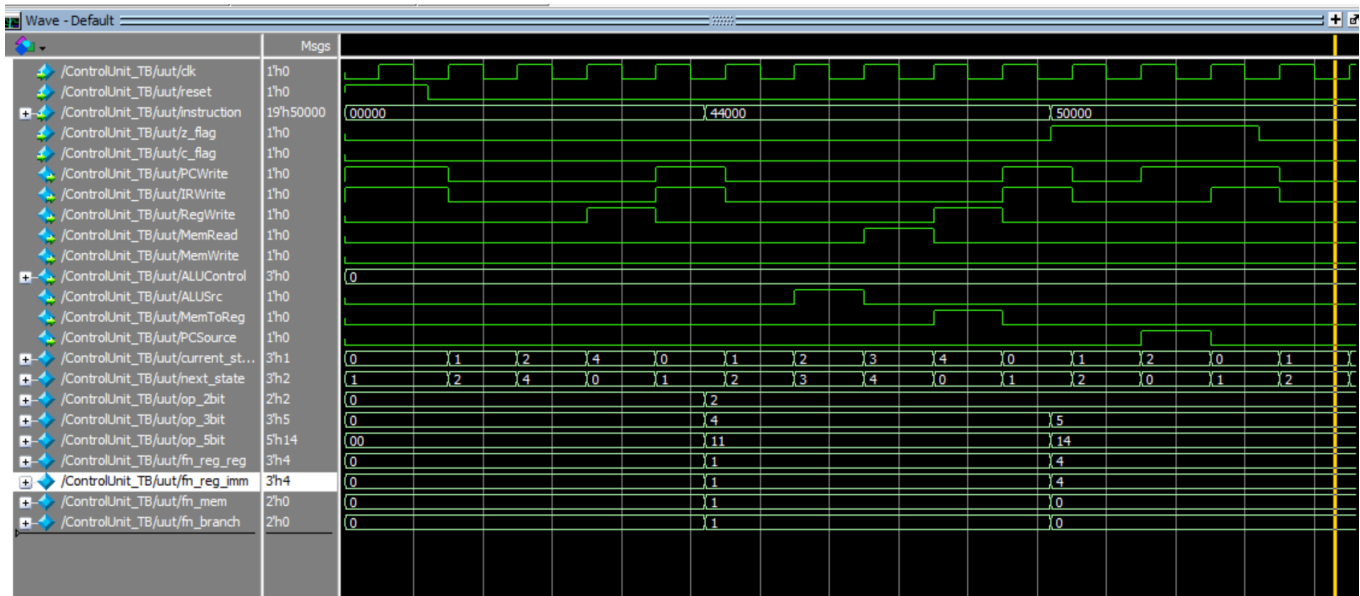
WRITEBACK: begin
    RegWrite = 1;
    if (op_3bit == 3'b100 && fn_mem == 2'b01) begin // If it was a Load
        MemToReg = 1; // Write data from memory to register
    end else begin
        MemToReg = 0; // Write data from ALU to register
    end
    next_state = FETCH; // Instruction complete!
end

default: next_state = FETCH;

EXECUTE: begin
    if (op_2bit == 2'b00) begin // ALU Reg-Reg
        ALUSrc = 0; // Use Reg2
        ALUControl = fn_reg_reg;
        next_state = WRITEBACK;
    end
    else if (op_2bit == 2'b01) begin // ALU Reg-Imm
        ALUSrc = 1; // Use 8-bit Immediate
        ALUControl = fn_reg_imm;
        next_state = WRITEBACK;
    end
    else if (op_3bit == 3'b100) begin // Memory (LDM / STM)
        ALUSrc = 1; // Address is r1 + disp (Immediate)
        ALUControl = 3'b000; // ADD to calculate address
        next_state = MEM_ACCESS;
    end
    else if (op_3bit == 3'b101) begin // Branch
        // Check flags to see if we should take the branch
        if ((fn_branch == 2'b00 && z_flag == 1) || // BZ
            (fn_branch == 2'b01 && z_flag == 0) || // BNZ
            (fn_branch == 2'b10 && c_flag == 1) || // BC
            (fn_branch == 2'b11 && c_flag == 0)) // BNC
        begin
            PCWrite = 1;
            PCSrc = 1; // PC = Branch Target
        end
        next_state = FETCH;
    end
    else if (op_5bit == 5'b111100) begin // Jump
        PCWrite = 1;
        PCSrc = 1; // PC = Jump Target
        next_state = FETCH;
    end
    else begin
        next_state = FETCH; // Fallback
    end
end
```

WAVEFORM:

- **ADD - 00000:** The first instruction is an ADD. You can see the current_state count 0, 1, 2, 4. It skips state 3 (memory) because math doesn't need it. At state 4, RegWrite turns to 1 to save the answer.
- **LDM - 44000:** The second instruction is Load Memory. This is the longest path. The state counts all five steps: 0, 1, 2, 3, 4. In state 3, MemRead turns on to read the RAM. In state 4, MemToReg and RegWrite turn on to safely save the RAM data into a register.
- **Branch BZ - Condition True:** The third instruction (50000) is a Branch on Zero. Here, the z_flag input is 1. When the state reaches 2 (Execute), you can see PCWrite and PCSource both turn on (1) to jump to a new address. Then the state goes straight back to 0 to fetch the next code.
- **Branch BZ - Condition False:** The last part tests the exact same branch instruction, but we forced the z_flag to 0. When the state reaches 2, PCWrite and PCSource correctly stay at 0, ignoring the jump.



PROCESSOR

CODE:

```
reg [11:0] PC;
reg [18:0] IR;           // Instruction Register
reg carry_flag_reg;      // carry
reg zero_flag_reg;       // zero

// control unit -> output wires
wire PCWrite, IRWrite, RegWrite, MemRead, MemWrite, ALUSrc, MemToReg, PCSource;
wire [2:0] ALUControl;

wire [18:0] fetched_instruction;
wire [7:0] read_data1, read_data2, alu_result, mem_read_data;
wire alu_z, alu_c;
```

This module holds everything together, therefore only inputs are clock and register, other variables are inside wires, some are outputs of control unit (the right signals to be given to each module) and some wires carrying data within the module.

On the sequential logic, as the reset or clock hits our processor, we either set everything to 0, or decide what is our next program counter (either +1 or some branch). Then we fetch the next instruction and reclaim the flags (carry and zero).

Then we have 3 multiplexers:

For the rest we just define all modules to connect.

```
// ALUSrc MUX? use Register 2, or immediate
wire [7:0] alu_operand2 = ALUSrc ? IR[7:0] : read_data2;

// MemToReg MUX: save the ALU math result, or Data Memory result?
wire [7:0] write_back_data = MemToReg ? mem_read_data : alu_result;

// Normally, read_reg2 is bits [7:5]. But for Store (STM), we need to read the
// data inside [13:11] so we can push it into Memory.
wire [2:0] read_addr_2 = MemWrite ? IR[13:11] : IR[7:5];
```

WAVEFORM:

This waveform shows the whole processor running a 4-line program. To test it, we first put 10 (which is 0a in hex) inside the RAM at address 5 before booting up.

1. On the first instruction (Load Memory, 1000100100000000101), the ALU calculates the address $0 + 5 = 5$. You can see `alu_result` is 05. Then `MemRead` goes to 1 to read the data, and `write_back_data` becomes 0a, saving 10 into register 1 (r1).
2. Then we have the Add instruction (00000010001001000000). It adds r1 to r1 and saves it to r2. Since r1 holds 10 (0a), $10 + 10 = 20$ (which is 14 in hex). In the wave, `alu_result` and `write_back_data` both correctly show 14.
3. For the third one, Store Memory (1001001000000000110), we need to save the answer back to RAM. The ALU calculates address 6. Then `MemWrite` turns to 1 so we can push the 14 from r2 inside memory address 6.
4. Finally we just define a Jump (11100000000000000000). At the very end of the picture, `PCSource` turns 1 to force the PC back to 000, so the loop starts all over again.

